The project life cycle and methodology

Life cycles describe how a project moves from idea to closure; methodologies prescribe the practices used along the way. In this chapter you learn to choose and tailor an approach for a given IT project.

PMBOK Guide 7th ed. | Scrum Guide 2020 | PRINCE2 7 | Agile Practice Guide

AGENDA

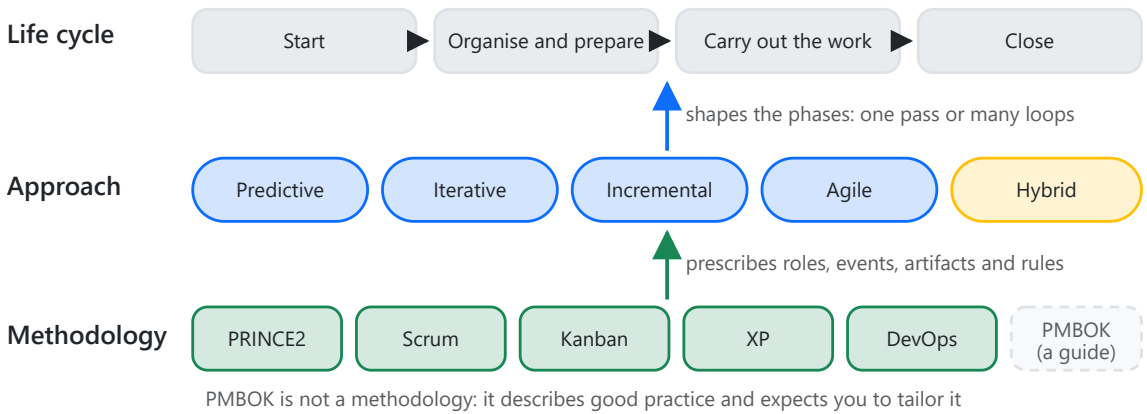
What we will cover

1. Generic life cycle phases and phase gates
2. Predictive (waterfall), iterative, incremental, agile and hybrid life cycles
3. Software development life cycle models: waterfall, V-model, spiral, RUP
4. Scrum: roles, events, artifacts, definition of done
5. Kanban, XP practices and DevOps as delivery approaches
6. PRINCE2 principles, themes and processes compared with PMBOK
7. Selecting an approach: uncertainty, requirements stability, team, contract
8. Tailoring: combining a predictive plan with agile delivery
9. Life cycle in practice: milestones, releases, sprints and stage reviews
10. Project closure in each model: acceptance, lessons learned, hand-over
11. Chapter Quiz
12. Lab-03

Click any item to jump directly to that topic.

Three words that are often confused

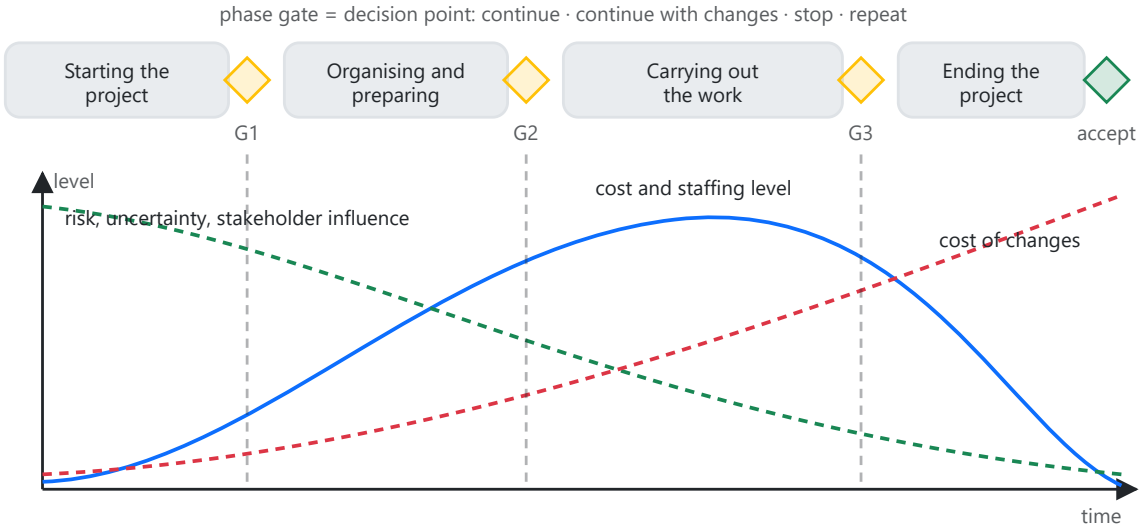
- **Project life cycle**: the series of phases a project passes through from start to completion (PMBOK Guide 7th ed.). It answers *when* things happen.
- **Development approach**: how the product is created and delivered: predictive, adaptive (iterative, incremental, agile) or hybrid. It shapes the life cycle.
- **Methodology** or **framework**: a system of practices, roles, events and rules used along the way: PRINCE2, Scrum, Kanban, XP, DevOps.
- Chapters 01 and 02 gave you the vocabulary and the constraints. This chapter gives you the **route**: which phases, which approach, how to choose, tailor, govern and close.
- For the Realtime Quiz project, Lab-03 ends with a **tailoring decision record** and a hybrid life cycle over the 14-week semester.



TOPIC 1

The generic life cycle: phases, gates, and the shape of the curves

- A **phase** is a collection of logically related activities that ends with one or more deliverables (PMBOK Guide 7th ed.).
- A **phase gate** (stage gate, kill point) is a review at the end of a phase: *continue*, *continue with changes*, *stop*, or *repeat the phase*. The decision belongs to the sponsor or steering committee, not to the team.
- **Cost and staffing** are low at the start, peak while the work is carried out and drop at closure.
- **Risk and uncertainty** (and stakeholder influence) are highest at the start and fall as decisions are made.
- The **cost of a change** rises over time: a requirement changed in week 2 costs a conversation; the same change in week 12 costs a rewrite.
- Every approach on the next slides is a way of bending these curves: adaptive approaches keep the cost of change low by deciding late and in small pieces.



Worked example: four gates for the Realtime Quiz project

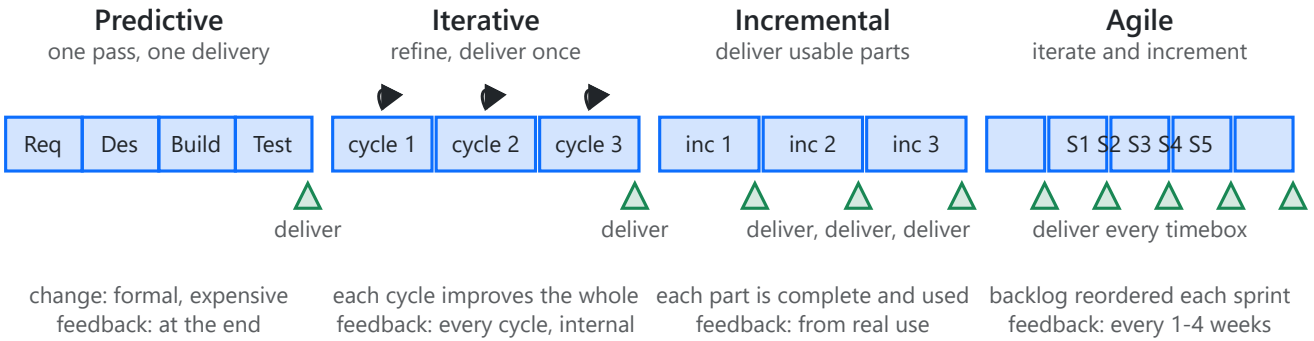
Gate	When	Evidence required	Decides
G1 plan approved	end of week 3	charter, scope statement, WBS, milestone schedule, budget in hours	sponsor
G2 MVP checkpoint	end of week 7	all Must features demoed end to end, CI green	sponsor + PM
G3 release candidate	end of week 11	coverage ≥ 80 %, load test 100 students, security checklist	sponsor + PM
G4 acceptance	week 14	pilot with 50 students without error, hand-over done	sponsor

💡 A gate is only useful if **stop** is a real option. If nobody would ever stop the project at G2, G2 is a status meeting, not a gate.

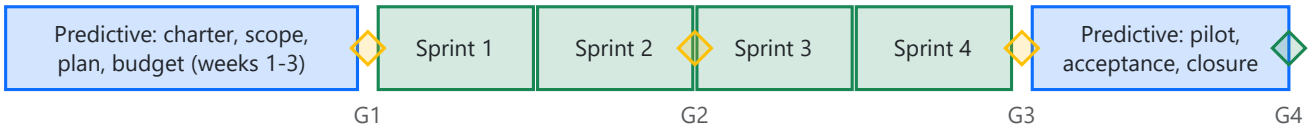
```
GATE REVIEW RECORD
Gate:       G2 · MVP checkpoint        Date: 2026-11-16 (end of week 7)
Chair:      Sponsor (Head of IT)       Presenter: project manager
Evidence checked
  [x] Must features demoed end to end on a laptop and a phone
  [x] ./mvnw verify green in CI, line coverage 71 % (80 % due at G3)
  [ ] Load test with 100 simulated students          -> moved to G3
  [x] Risk register reviewed: 2 new risks, 1 closed (proxy blocks WS)
  [x] Hours used 168 of 320 planned (52 %), schedule on track
Decision:   CONTINUE WITH CHANGES
Conditions: 1. move "chat" (Could) out of sprint 3
            2. add a reconnect-and-resume test before G3
Next gate:  G3 release candidate, 2026-12-14
Signed:     sponsor, project manager
```

Five life cycle shapes: how often do you loop, how often do you deliver?

- **Predictive**: scope, time and cost fixed early; one pass through requirements, design, build, test; one delivery at the end. Change goes through formal change control.
- **Iterative**: the same product is refined in repeated cycles (prototype, review, improve); delivery still at the end. Good when the *solution* is unclear.
- **Incremental**: usable subsets are delivered one after another; each increment adds features. Good when *early value* matters.
- **Agile**: both iterative and incremental, in short fixed timeboxes, with the customer reviewing every increment and reordering the backlog.
- **Hybrid**: a deliberate mix, for example a predictive plan and gates around agile delivery. (Agile Practice Guide, PMBOK Guide 7th ed.)



Hybrid (this course's choice for the Realtime Quiz project)



gates give the sponsor control; sprints give the team feedback and absorb change

Same product, different route: what changes for the sponsor and the team

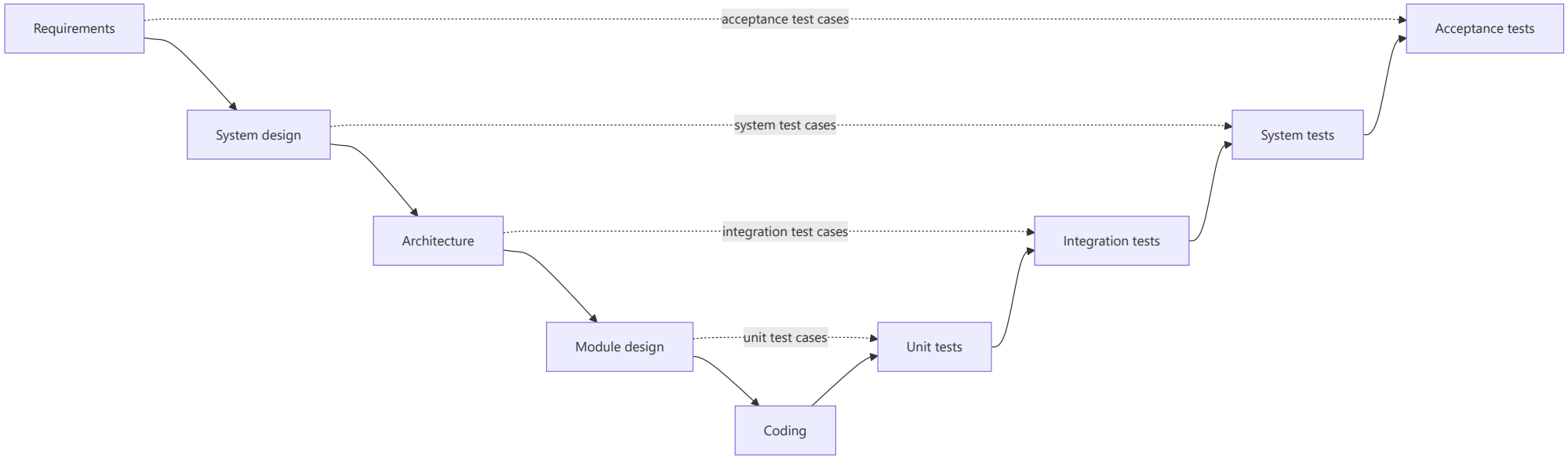
	Predictive	Iterative	Incremental	Agile
Requirements	fixed up front	elaborated per cycle	fixed, delivered in parts	dynamic, reordered
Delivery	once, at the end	once, at the end	several planned releases	every timebox
Change	change control board	absorbed in next cycle	between increments	backlog refinement
Risk handled by	planning and gates	prototypes	early value	short feedback loops
Typical contract	fixed price	time and materials	per release	time and materials, capped
Fits when	stable, regulated, known tech	solution unclear	value needed early	requirements emerge, customer available

```
WORKED EXAMPLE · Realtime Quiz Must features, three routes
Team: 4 students + lecturer as PM; 14 weeks; pilot in week 14

Route         Sponsor first plays  Reviews   Change in W9   Risk
              a working quiz       before    "join code     found
                                   pilot     must be QR"    late?
------------  -------------------  --------  -------------  ------
Predictive    week 12 (system      0         reject or      yes
              test)                          replan W12-13
Incremental   week 7 (release 1:   2         goes into      medium
              accounts+authoring)            release 3
Agile/hybrid  week 5 (sprint 1     4         backlog item   no
              review)                        for sprint 4

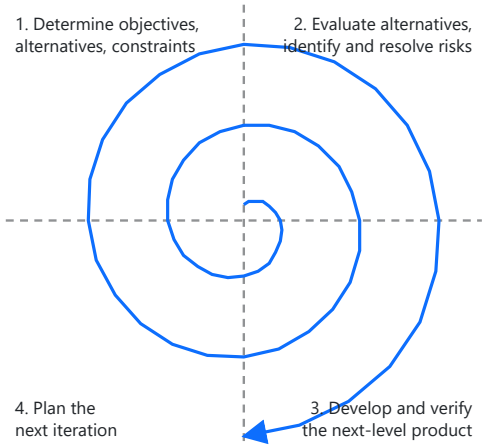
Same 8 Must features, same team: only the order and the
number of feedback points differ. Count the reviews: that
is the number of chances to be wrong cheaply.
```

SDLC models: waterfall and the V-model



- **Waterfall** (Royce, 1970): requirements, design, implementation, verification, maintenance, each phase completed and documented before the next. Royce himself warned that a single pass fails without feedback to the previous phase.
- **V-model**: the waterfall bent into a V. Each specification level on the left arm has a matching **test level** on the right arm, and the test cases are written *when the specification is written*, not after coding.
- Still used for safety-critical and regulated software (medical, automotive, public tenders) where traceability from requirement to test is mandatory.
- Realtime Quiz mapping: FR-A, FR-Q, FR-S, FR-C, FR-R → acceptance tests in the pilot session; pages, API and STOMP events → end-to-end system tests; controllers, services, repositories → Spring Boot integration tests with H2; ScoringService, SessionRegistry → JUnit and Mockito unit tests.

Risk-driven models: the spiral and the Rational Unified Process



Spiral model (Boehm, 1988)

Each loop is one iteration through the four quadrants; the radius is the cumulative cost. Risk decides what the next loop builds: a prototype, a benchmark, a design or a release.

RUP (Rational, 1998)

Inception	Elaboration	Construction	Transition
	LCO	LCA	IOC release

Phases hold several iterations; every iteration runs all disciplines (require-ments, design, implementation, test).

Milestones: LCO scope agreed, LCA architecture proven, IOC usable release.

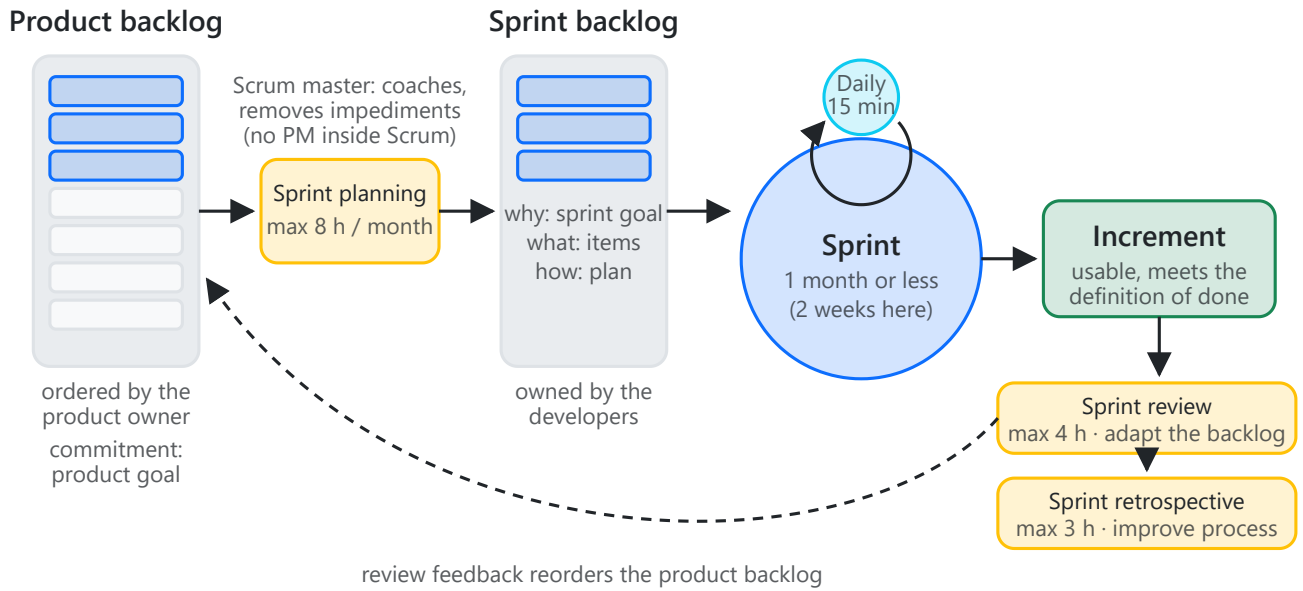
Iterative at heart, use-case driven, architecture-centric.

```
SPIRAL LOOP 2 · Realtime Quiz, "real-time broadcast" risk
1. Objectives     every client sees a question within 500 ms;
                  works from the campus network
   Alternatives   a) STOMP over WebSocket, simple broker
                  b) STOMP relay to RabbitMQ
                  c) HTTP polling every second
   Constraints    one VPS (2 vCPU, 2 GB), 4 students, 2 weeks
2. Risks          campus proxy blocks WebSocket (high);
                  RabbitMQ adds operations work (medium);
                  polling loads the server (high at 100 users)
   Resolution     prototype a) with SockJS fallback, measure
                  latency with 100 simulated clients
3. Develop/verify prototype: 180 ms median, fallback works
4. Plan next loop choose a); sprint 2 builds the session core;
                  keep b) as the documented scaling path
```

RUP's **Elaboration** phase is the same idea as spiral loops 1-2: prove the architecture on the riskiest use cases before you commit the whole team to Construction.

Scrum: three accountabilities, five events, three artifacts Scrum Guide 2020

- **Product owner**: maximises value; owns and orders the product backlog; one person, not a committee.
- **Scrum master**: makes Scrum work; coaches the team and removes impediments; not a project manager.
- **Developers**: everyone who builds the increment; they own the sprint backlog and the plan for the sprint.
- **Events**: the sprint (a container of one month or less), sprint planning, daily scrum, sprint review, sprint retrospective. Every event is timeboxed.
- **Artifacts** and their commitments: product backlog → product goal; sprint backlog → sprint goal; increment → definition of done.
- Scrum has no "project manager". In a hybrid project the PM works *around* the Scrum team: gates, budget, stakeholders, risks.



Worked example: timeboxes and the sprint 1 plan for the Realtime Quiz team

Event	Scrum Guide max (1-month sprint)	Our 2-week sprint
Sprint	1 month	2 weeks, Tue to Mon
Sprint planning	8 h	2 h, first day 10:00
Daily scrum	15 min	15 min, 09:00, online
Sprint review	4 h	1 h, last day 14:00, sponsor invited
Sprint retrospective	3 h	45 min, last day 15:15
Backlog refinement	not an event	30 min, mid-sprint Thursday

⚠ A sprint goal is one sentence the sponsor understands, not a list of tickets. If everything in the sprint is "the goal", nothing can be dropped when the team runs late.

```
SPRINT 1 PLAN · weeks 4-5 (2026-10-20 to 2026-11-02)
Sprint goal: an instructor can register, log in and author a quiz
Capacity:    4 students x 10 h/week x 2 weeks = 80 h
Velocity:    unknown (first sprint); planning guess 30 points

ID    Backlog item (all Must)                       FR         Pts
B-01  Project skeleton, CI, Docker Compose, Flyway  -            3
B-02  Register with email, name, password, role     FR-A1        3
B-03  Log in and receive a JWT (12 h expiry)        FR-A2, A3    5
B-04  Role checks: instructor and student pages     FR-A2        3
B-05  Create a quiz: title, description, visibility FR-Q1        5
B-06  Add, edit, reorder, delete questions          FR-Q2, Q3    8
B-07  Single-answer and multi-answer questions      FR-Q4        3
                                          Committed:            30
Not in sprint 1: FR-S* (live sessions) -> sprint 2, per the roadmap
Risk watched:   Spring Boot 4 examples outdated (mitigation: docs)
```

Worked example: definition of done for the Realtime Quiz team

```
## Definition of Done (applies to every backlog item, every sprint)
1. Code merged to `main` through a merge request reviewed by
   another student (no self-merge).
2. Unit and integration tests written; `./mvnw verify` green in CI.
3. Line coverage of new `service` and `realtime` code >= 80 % (JaCoCo).
4. New REST endpoints appear in the OpenAPI document (`/v3/api-docs`).
5. New STOMP events are listed in the WebSocket table of the plan.
6. Flyway migration applies cleanly on PostgreSQL and MySQL profiles.
7. Feature demonstrated in a browser on a desktop and a phone viewport.
8. No secrets in the repository; no `high` finding in the dependency scan.
9. User-visible change noted in `CHANGELOG.md`.

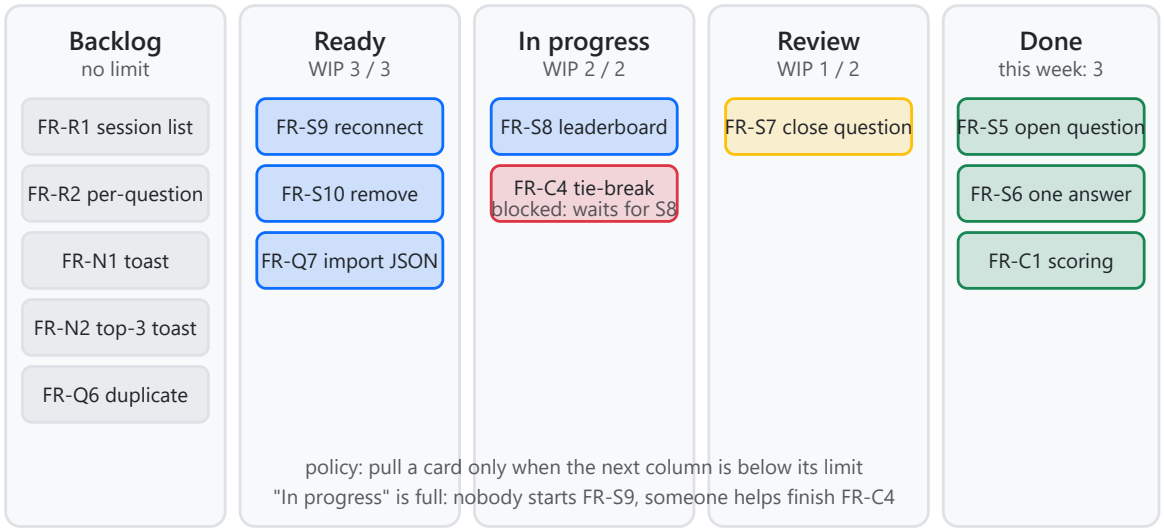
## Definition of Ready (an item may enter sprint planning when)
- It has an FR id from the Realtime Quiz plan and a MoSCoW priority.
- Acceptance criteria are written (Given / When / Then, 2-5 lines).
- It is estimated at 8 points or less (otherwise split it).
- Dependencies on other items or on the VPS are known.
- A page sketch exists if the item adds or changes a screen.
```

	Definition of done	Acceptance criteria
Scope	every item, every sprint	one backlog item
Owner	whole Scrum team (organisation may set a minimum)	product owner
Answers	"is our work releasable?"	"does it do what was asked?"
Example	coverage ≥ 80 %, reviewed, demoed	"a wrong join code shows an error within 1 s"

The Scrum Guide 2020 says work that does not meet the definition of done **cannot be released** and goes back to the product backlog; it is not "90 % done".

⊗ Anti-pattern: a DoD item like "the leaderboard sorts by score" belongs to *one* story. Keep the DoD generic, keep acceptance criteria specific.

Kanban: visualise the work, limit work in progress, manage flow



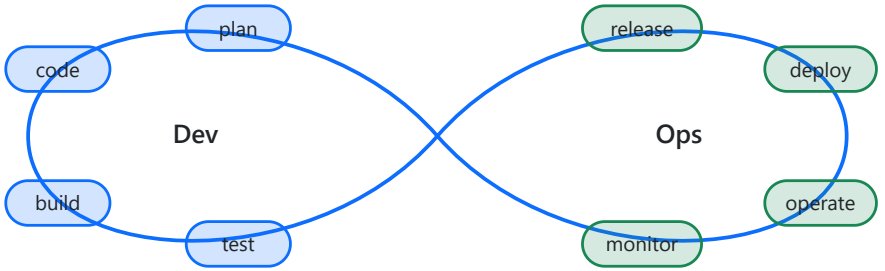
- Kanban is a **flow** method, not timeboxed: no sprints, work is pulled when capacity frees up. Practices: visualise, **limit WIP**, manage flow, make policies explicit, feedback loops, improve together.
- Metrics: **lead time** (request to done), **cycle time** (start to done), **throughput** (items per week), cumulative flow diagram.
- Fits support, operations and the hardening weeks of a project; many teams run Scrum with a Kanban board inside the sprint.

```
LITTLE'S LAW   lead time = WIP / throughput
Board above:   WIP = 6 cards in Ready + In progress + Review
               throughput = 3 cards done per week
               lead time = 6 / 3 = 2 weeks per card
Cut WIP to 3:  lead time = 3 / 3 = 1 week, same team
=> limiting WIP shortens the wait, not the work
```

XP gives the engineering practices, DevOps closes the loop to operations

XP practice (Beck)	In the Realtime Quiz project
Test-driven development	ScoringService tests written before the formula
Pair programming	WebSocket concurrency code always in pairs
Continuous integration	GitHub Actions runs `./mvnw verify` on every push
Small releases	deploy to the VPS at the end of every sprint
Refactoring, simple design	no "future-proof" broker until scaling is needed
Collective ownership, coding standard	anyone may change any file; Checkstyle in the build
On-site customer, sustainable pace	lecturer answers within a day; 10 h/week per student

continuous integration → continuous delivery → continuous deployment

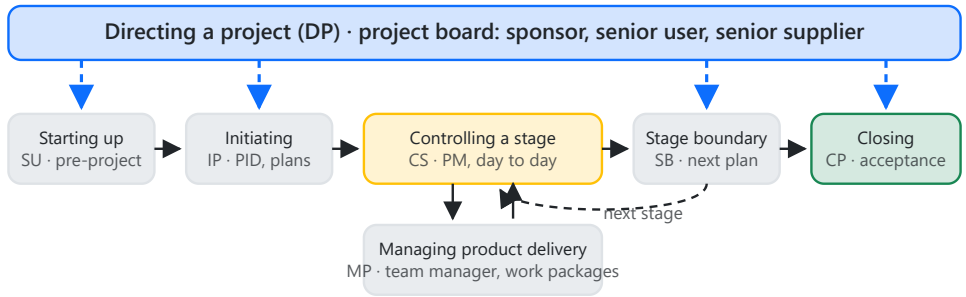


Actuator metrics and incident reports flow back into the backlog

```
PIPELINE (GitHub Actions, one job per stage)
push to main -> build+test: ./mvnw verify (JUnit, JaCoCo >= 80 %)
             -> package:    docker build -t quiz:sha
             -> deploy:     ssh gicitc.com docker compose up -d
             -> verify:     curl /actuator/health == UP
             -> monitor:    open sessions gauge, error rate
```

PRINCE2: seven principles, seven practices, seven processes PRINCE2 7

Principles (why)	Practices (what; "themes" before PRINCE2 7)	Processes (when)
Continued business justification	Business case	Starting up a project (SU)
Learn from experience	Organising	Directing a project (DP)
Defined roles, responsibilities, relationships	Quality	Initiating a project (IP)
Manage by stages	Plans	Controlling a stage (CS)
Manage by exception	Risk	Managing product delivery (MP)
Focus on products	Issues	Managing a stage boundary (SB)
Tailor to suit the project	Progress	Closing a project (CP)



Manage by exception: the board sets *tolerances* (time, cost, scope, quality, risk, benefits, sustainability). The PM decides inside them and escalates only when a forecast breaches one.

PRINCE2 processes mapped to the PMBOK process groups, and a stage plan outline

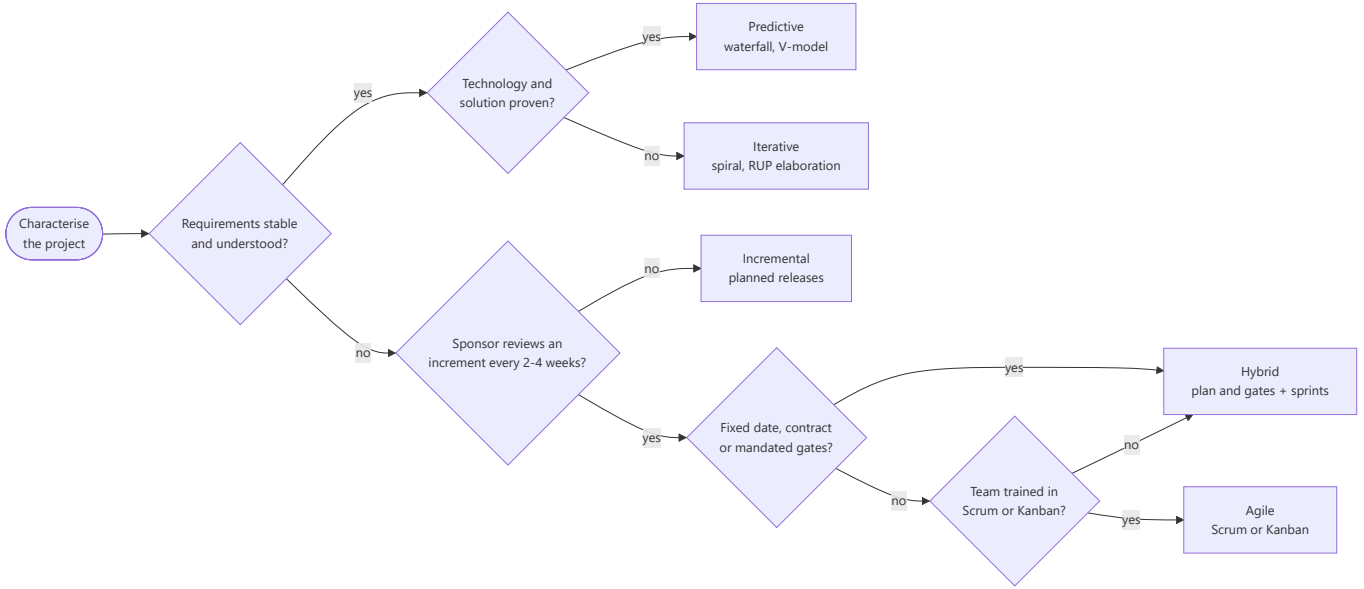
PMBOK process group	PRINCE2 process(es)	Key difference
Initiating	Starting up a project; DP authorises initiation	PRINCE2 needs a project brief and an outline business case before any planning
Planning	Initiating a project; Managing a stage boundary	PRINCE2 plans in detail one stage at a time (product-based planning)
Executing	Managing product delivery	work packages handed to a team manager, who may run Scrum inside
Monitoring and controlling	Controlling a stage; DP gives ad hoc direction	tolerances and exception reports replace continuous oversight
Closing	Closing a project	end project report, lessons report, follow-on action recommendations, benefits review plan

	PMBOK Guide 7th ed.	PRINCE2 7
Nature	descriptive guide: principles, performance domains, tailoring	prescriptive method: roles, documents, processes
Decisions	project manager, sponsor	project board; PM manages by exception
Certification focus	PMP (experience-based)	Foundation, Practitioner (method-based)

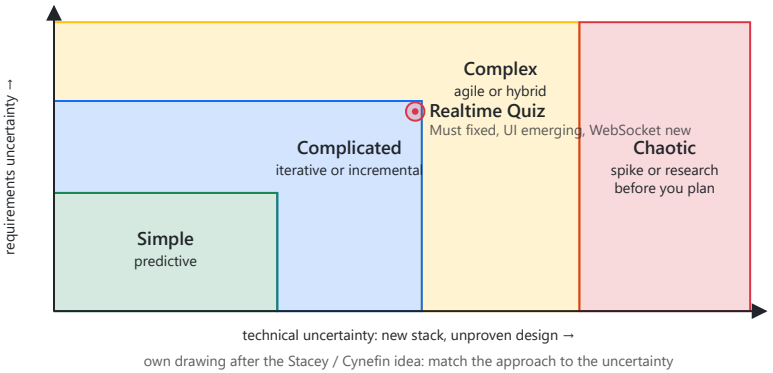
```
# Stage plan · Stage 2 "MVP delivery" (weeks 4-7)
Objective: Must features working end to end (gate G2, 2026-11-16)
Products (product descriptions in Lab-04):
- Accounts and security (FR-A1..A4)
- Quiz authoring (FR-Q1..Q5)
- Live session core (FR-S1..S7)
Schedule: sprint 1 (W4-5), sprint 2 (W6-7); end stage assessment W7
Resources: 4 students x 10 h/week x 4 weeks = 160 h; lecturer 2 h/week
Tolerances (manage by exception):
- time  +1 week          - cost  +10 % of hours
- scope no Must dropped   - quality coverage >= 70 % at G2
Controls: weekly highlight report (Friday); sprint review =
  checkpoint; exception report if a tolerance is forecast to fail
Risks carried: campus proxy blocks WebSocket (SockJS prototype)
Escalation: PM -> project board (sponsor, senior user, senior supplier)
```

Selecting an approach: a decision flow built from four questions

- **Requirements stability**: can they be written down now and stay valid? If not, plan to learn.
- **Uncertainty of the solution**: proven technology and a team that has built this before, or new ground?
- **Team and customer**: co-located, trained in agile, and is a product owner available every week?
- **Contract and governance**: fixed price, mandated stage gates, audits, a fixed date?
- PMBOK Guide 7th ed. groups these as product, project and organisation factors; the Agile Practice Guide calls the same idea a *suitability filter* (culture, team, project).
- The flow gives a *starting point*. The scoring on the next slide makes the choice explicit and defensible.



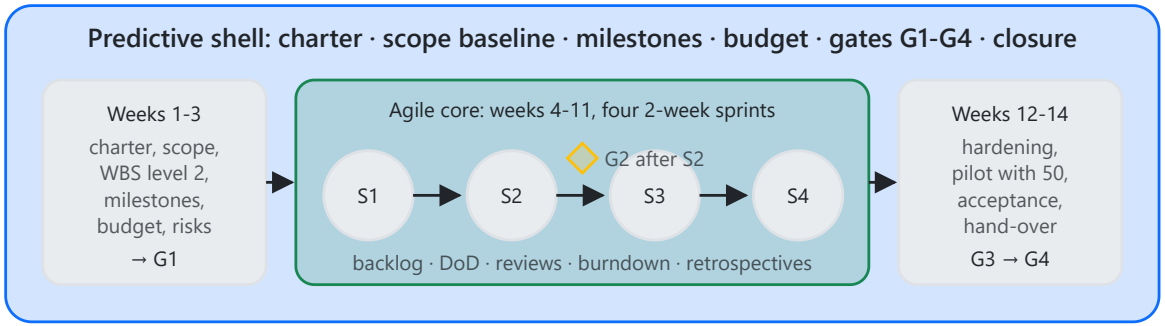
Worked example: where the Realtime Quiz project sits, and a weighted score



own drawing after the Stacey / Cynefin idea: match the approach to the uncertainty

```
WEIGHTED SCORING (fit 1 = poor .. 5 = excellent), weights sum to 100
                                  Weight  Waterfall  Scrum  Hybrid
Requirements stability (Must fixed)    25        3        4       5
Technical uncertainty (WS, Boot 4)     20        2        5       4
Team experience (first Scrum team)     20        3        2       4
Governance: fixed date, sponsor gates  20        4        2       5
Early delivery and feedback (pilot)    15        1        5       4
Weighted score = SUM(weight x fit) / 100
  Waterfall = (25x3 + 20x2 + 20x3 + 20x4 + 15x1) / 100 = 270/100 = 2.70
  Scrum     = (25x4 + 20x5 + 20x2 + 20x2 + 15x5) / 100 = 355/100 = 3.55
  Hybrid    = (25x5 + 20x4 + 20x4 + 20x5 + 15x4) / 100 = 445/100 = 4.45
Spreadsheet: C8 = SUMPRODUCT($B$2:$B$6,C2:C6)/100   (copy to D8, E8)
Decision: hybrid. Sensitivity: Scrum wins only if governance weight
drops below 5 and team experience is scored 4 or more.
```

Tailoring: a predictive shell around an agile core



Tailoring steps (PMBOK Guide 7th ed.)

1 Select initial approach → 2 Tailor for the organisation → 3 Tailor for the project → 4 Implement ongoing improvement

Keep from predictive	Keep from agile	Drop
charter, scope statement, WBS to level 2	product backlog with FR ids	task-level Gantt for weeks 4-11
milestone schedule and gates	2-week sprints, reviews, retrospectives	change control board for backlog items
budget in hours, earned value at gates	definition of done, burndown	big design up front beyond the architecture
formal acceptance and closure	daily scrum, product owner	separate test phase for Must features

⚠ Tailoring is **deliberate** and **written down**: every practice you keep, add or drop needs a reason. "We do Scrum but skip the retrospective" is not tailoring, it is drift.

Worked example: the tailoring decision record (ADR style)

```
# TDR-01 · Life cycle and methodology for the Realtime Quiz project
Status: accepted · 2026-10-16 · decided by the sponsor and the PM

## Context
14-week semester, 4 students plus the lecturer as project manager.
The sponsor (Head of IT) wants an approved plan and budget by week 3
and a pilot with 50 students in week 14. The Must features are fixed
by the charter; screen design and real-time details will emerge.
The team has not used Scrum before; Spring Boot 4 is new to them.

## Options considered
1. Waterfall: design complete by W5, build W6-11, test W12-13.
2. Pure Scrum: six sprints from W2, no baseline plan, no gates.
3. Hybrid: predictive W1-3 and W12-14, four 2-week sprints W4-11.

## Decision
Option 3 (weighted score 4.45 vs 3.55 and 2.70). Gates G1-G4 give the
sponsor control points; sprints give feedback every two weeks and
absorb UI change without a change board.

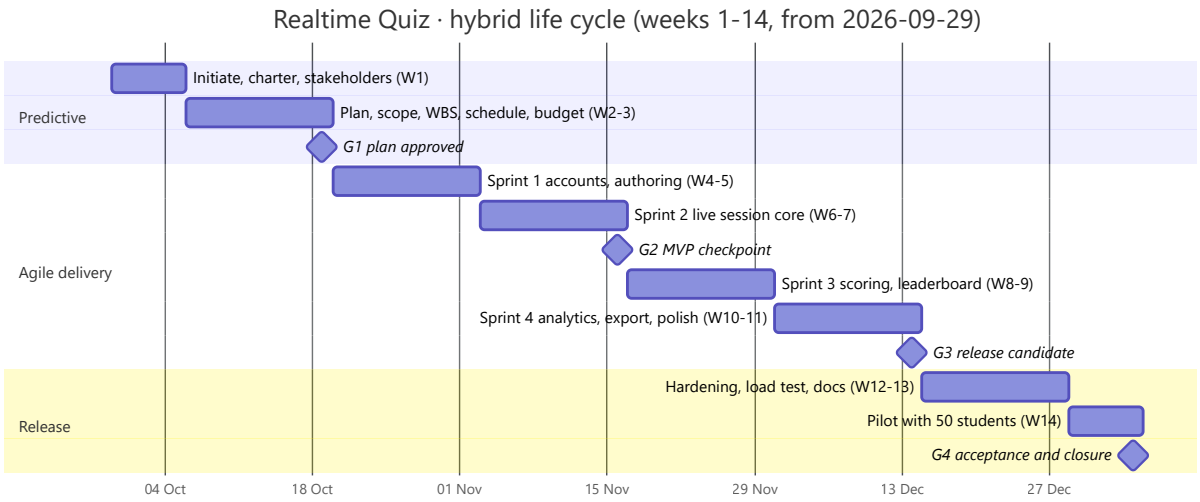
## Consequences
+ playable quiz at the sprint 1 review (W5), MVP proven at G2 (W7)
+ fixed deadline protected by G3; hours tracked against the budget
- two vocabularies (stage vs sprint): glossary added to the plan
- product owner (student) and PM (lecturer) are different people:
  decision rights written in the RACI (Lab-08)
- first sprint velocity is a guess: re-plan sprints 2-4 after S1
```

Section	Must answer
Context	the facts that constrain the choice: time, team, sponsor, uncertainty
Options	at least three, each realistic, described in one line
Decision	one option, with the evidence (scores, flow) that led to it
Consequences	both the gains and the new problems you accept, with a mitigation each

The record is short on purpose: one page that a new team member reads in five minutes and that the sponsor signed. It is versioned with the plan; a change of approach means a new record, not an edit.

Lab-03 Task 3 asks you to write exactly this record for your own scoring from Tasks 1-2, and to draw the chosen life cycle.

The hybrid life cycle on the calendar: 14 weeks, four gates, four sprints

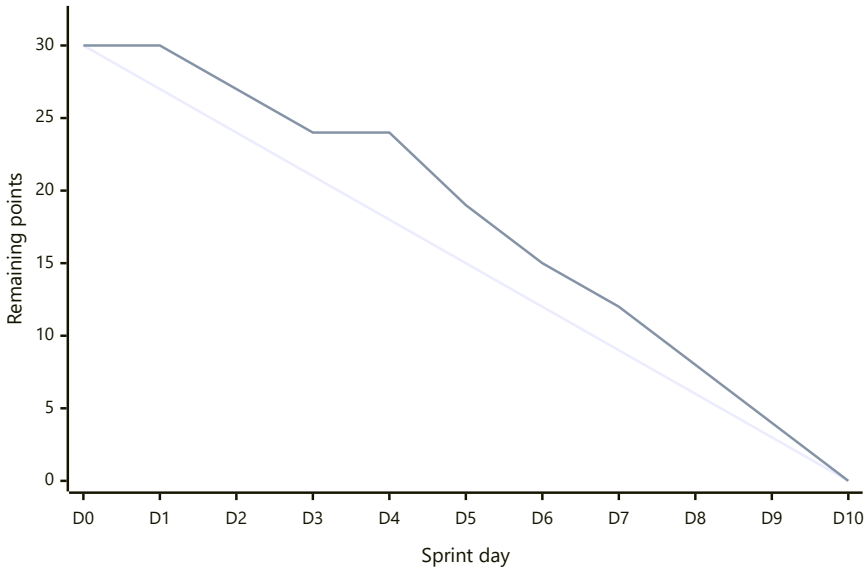


Control point	Meaning	Here
Milestone	a point in time, zero duration, something is true	G1-G4
Release	a version handed to users	R1 internal (W7), R2 candidate (W11), R3 pilot (W14)
Sprint	a timebox that ends with a review	S1-S4, 2 weeks each
Stage review	gate meeting with the sponsor: continue, change, stop	G1-G4, chaired by the sponsor

A sprint review is *not* a gate: the team and the product owner inspect the increment. The gate is where the **sponsor** decides about money and time. In this plan G2 happens right after the sprint 2 review, same afternoon.

Inside a sprint: reading the burndown, and what the gates see

Sprint 1 burndown: ideal vs actual (story points remaining)



first line: ideal · second line: actual, updated after each daily scrum

```
READING THE BURNDOWN (sprint 1, 30 points, 10 working days)
ideal burn rate   = 30 / 10 = 3 points per day
day 4: ideal 18, actual 24  -> 6 points behind (two days of work)
needed from day 4 = 24 / 6 remaining days = 4 points per day
day 7: ideal 9, actual 12   -> 3 behind, recovering
day 10: 0 remaining         -> sprint goal met, velocity = 30

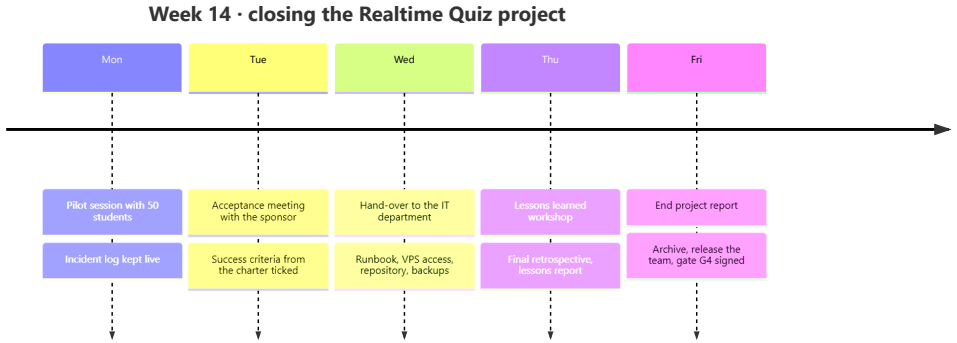
Forecast for the remaining Must + Should backlog:
backlog 120 points / velocity 30 = 4 sprints = 8 weeks
=> fits weeks 4-11 exactly; any new Must item pushes a Should out

Mermaid source pattern (Lab-03 challenge):
  x-axis "Sprint day" [D0, D1, ..., D10]
  line [ideal values]      line [actual values]
```

⚠ A flat line for two days (D0-D1, D3-D4 above) is the signal to look for: items too big, a blocker, or a student absent. Ask at the daily scrum, not at the review.

Project closure: acceptance, lessons learned and hand-over in each model

Model	Acceptance	Lessons learned	Hand-over
Predictive / waterfall	formal sign-off against the scope statement at the end	one post-project review; report filed	documentation package to operations, warranty period
V-model	acceptance tests traced to every requirement	defect statistics per test level	test evidence and traceability matrix delivered
Scrum / agile	every sprint: increment meets the definition of done, reviewed by the product owner	every sprint retrospective; final one for the project	continuous: DevOps pipeline, product owner keeps the backlog
PRINCE2	Closing a project: confirm product acceptance, verify benefits plan	lessons log throughout, lessons report at closure	follow-on action recommendations; operational hand-over confirmed
Hybrid (ours)	DoD each sprint and G4 sign-off after the pilot	four retrospectives plus a closure workshop	repository, VPS, runbook to the IT department



PMBOK Guide 7th ed.: a project also closes when it is **cancelled**. Then acceptance is replaced by a record of what was delivered and why it stopped; lessons learned matter even more.

Worked example: closure checklist and lessons learned register

```
## Closure checklist · Realtime Quiz (gate G4, week 14)
### Acceptance
- [ ] Pilot ran with >= 50 students, no severity-1 incident
- [ ] Every Must and Should requirement verified (test evidence linked)
- [ ] Sponsor signed the acceptance form against the charter criteria
### Hand-over
- [ ] Repository transferred to the department's GitHub organisation
- [ ] VPS credentials rotated and stored in the department vault
- [ ] Runbook: deploy, backup, restore, rotate JWT secret, on-call
- [ ] Known issues and the open backlog exported (CSV) to the owner
### Lessons learned and administration
- [ ] Lessons learned workshop held; report filed in `docs/closure/`
- [ ] Final hours and cash compared with the budget; variance explained
- [ ] Contracts closed: VPS renewal decided, domain owner recorded
- [ ] Team released; individual feedback given; grades submitted
- [ ] Project archive tagged `v1.0-closed`; plan marked CLOSED
```

#	What happened	Effect	Recommendation	Owner
L1	Sprint 1 velocity guessed at 30, actual 30	plan held	keep first-sprint guess at 7-8 points per student	PM
L2	Campus proxy blocked WebSocket in the pilot room	SockJS fallback used, 2 s lag	test in the real room at G3, not only on the VPS	team
L3	Chat (Could) pulled into sprint 3 by a student	FR-S9 slipped to sprint 4	product owner alone reorders; PM checks at the review	PO
L4	Gate G2 record took a day to write	late report	fill the gate template during the sprint review	PM

☑ A lesson is only learned when it names a **recommendation and an owner**. "Communication could be better" is a feeling, not a lesson.

BEST PRACTICES

Best practices and common mistakes when choosing and running a life cycle

1 Decide, record, sign

Write the tailoring decision record before week 3 and have the sponsor sign it. An approach nobody agreed to is renegotiated at every problem.

2 Gates with a real stop option

Define the evidence for each gate in advance and let the sponsor chair it. A gate that always says "continue" is a status meeting.

3 One definition of done

Keep the DoD generic, tested by CI, and visible on the board. Put story-specific checks in the acceptance criteria instead.

Mistake: "Scrum-fall"

Sprints that only code, followed by a big test phase, give the ceremonies of agile with the risk profile of waterfall. Test inside the sprint or call it incremental.

Mistake: no baseline in a hybrid

Agile delivery does not remove the need for a milestone schedule and a budget when the date and the sponsor are fixed. Without a baseline, G3 cannot judge anything.

Mistake: closure skipped

Teams that "just stop" after the last sprint leave no runbook, no acceptance record and no lessons. Book the closure week in the plan from day one.

Chapter quiz 10 questions

1. What is the purpose of a phase gate at the end of a life cycle phase?

- To let the team demonstrate the increment to the product owner
- A decision by the sponsor or steering committee to continue, continue with changes, stop or repeat the phase
- To update the burndown chart
- To assign story points to the next phase

2. Which life cycle delivers usable subsets of the product one after another, each adding features, without reworking the earlier parts?

- Predictive
- Iterative
- Incremental
- Spiral

3. In the V-model, which test level verifies the architecture design (controllers, services, repositories working together)?

- Unit tests
- Integration tests
- System tests
- Acceptance tests

4. Which statement about the definition of done is correct according to the Scrum Guide 2020?

- It is written per backlog item by the product owner
- It lists the features planned for the sprint
- It applies to every increment; work that does not meet it cannot be released and returns to the product backlog
- It is the same thing as the sprint goal

5. The remaining Must and Should backlog is 120 story points and the team's velocity is 30 points per two-week sprint. How many weeks of sprints are needed?

- 4 weeks
- 6 weeks
- 8 weeks
- 12 weeks

6. A Kanban board holds 8 cards in progress and the team finishes 4 cards per week. By Little's law, what is the average lead time of a card?

- 0.5 week
- 2 weeks
- 4 weeks
- 32 weeks

7. A sprint starts with 30 points over 10 days. On day 4 the ideal line shows 18 points remaining and the actual is 24. What is the daily rate needed to finish on time?

- 3 points per day
- 4 points per day
- 6 points per day
- 2 points per day

8. What does the PRINCE2 principle "manage by exception" mean in practice?

- The project manager reports every decision to the board
- The board sets tolerances; the PM decides inside them and escalates only when a tolerance is forecast to be exceeded
- Exceptions are logged in the issue register and ignored
- The team may skip a stage boundary if it is late

9. Two criteria carry weights 60 and 40. An approach scores 5 on the first and 2 on the second. What is its weighted score on a 1-5 scale?

- 3.5
- 3.8
- 4.2
- 7.0

10. In a hybrid life cycle such as the Realtime Quiz plan, when are lessons learned collected?

- Only in the closure week, so the team is not distracted
- At every sprint retrospective and again in a closure workshop at the final gate
- Only when the project is cancelled
- Never; lessons belong to the next project

WRAP-UP

Summary

A life cycle is the sequence of phases; a phase gate is a sponsor decision to continue, change, stop or repeat.

Predictive, iterative, incremental, agile and hybrid differ in how often you loop and how often you deliver; the cost of change decides which fits.

Waterfall and the V-model give traceability from requirement to test; spiral and RUP put the riskiest work in the first loops.

Scrum: product owner, scrum master, developers; sprint, planning, daily, review, retrospective; backlog, sprint backlog, increment with a definition of done.

Kanban limits work in progress (Little's law: lead time = WIP / throughput); XP supplies the engineering practices; DevOps closes the loop to operations.

PRINCE2 is a prescriptive method (7 principles, practices, processes, manage by exception); PMBOK is a descriptive guide to tailor.

Select an approach from requirements stability, uncertainty, team and contract; make it explicit with a weighted score.

Tailoring means a written decision record: what you keep from predictive, what from agile, what you drop, and why.

Every model closes with acceptance, lessons learned and hand-over; in agile they happen every sprint, in a hybrid also at the final gate.

Open Lab-03: 5 tasks + 1 challenge

Next chapter: 04 · Project Planning and Scope Management